Prerequisites and objectives

- Prerequisites are the Array ADT, index intervals, and loop invariants.
- Distinguish LIFO stacks from FIFO queues by their contracts. Handle empty, full, and wrapped states.
- Explain maze search and expression evaluation through state traces. A data structure choice alone does not specify a complete algorithm.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The LIFO Stack ADT

- The abstract state is $[a_0, \ldots, a_{n-1}]$, with the top at the end. `push(x)` appends x.
- `top()` returns the last element without changing the state. `pop()` removes it and requires a nonempty stack.
- The interface must define empty-stack failure, for example an optional result, exception, or error code. It must not silently access invalid memory.
- With a dynamic array, top and pop take constant time and push takes amortized constant time under constant-time element operations. A fixed-capacity version also defines a full-stack result.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Represent the stack top using n

- Maintain $0 \le n \le \text{capacity}$, with valid elements in $[0, n)$. The top is $A[n-1]$, accessible only when $n > 0$.
- For push, ensure capacity, write $A[n]$, then increment n. Do not increment n before allocation succeeds.
- For pop, check $n > 0$ before removing the last live object. Generic C++ must handle destruction as well as the index.
- Representing empty as n=0 avoids encoding top=−1 in an unsigned index.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Array stack operations and object lifetime

Let storage be uninitialized capacity C and n the number of live objects. The abstract sequence is storage[0..n). Allocating C slots does not construct C objects of type T.

```
top(): require n>0; return const T& to storage[n-1]
push(T value):           // snapshot constructed before entering the body
    if n==C: grow_to(max(1,2*C))
    construct_at(storage+n, value)   // commit n only after construction succeeds
    n=n+1
pop():
    require n>0
    destroy_at(storage+n-1)
    n=n-1
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

For C=2,n=0, push(8),push(3),push(2),pop() produce `(2,1,[8])`, `(2,2,[8,3])`, `(4,3,[8,3,2])`, `(4,2,[8,3])`. The third push moves two existing values first, so one push may cost Θ(n). Doubling gives Θ(n) total append work from empty.

Array stack operations and object lifetime (continued)

Generic T requires separately accounting for move, copy, and destruction costs. Preserving the old state on failed growth requires a suitable copy or noexcept-move strategy. Arbitrarily throwing moves do not automatically provide the strong exception guarantee.

Push(T value) takes an independent by-value parameter and assumes copy-constructible T. Although top() returns const T&, push(top()) copies that value before entering the body or growing. With capacity1,size1,and[7], construct value=7, relocate the old7, release old storage, construct element2 from value, then commit size2, producing[7,7]. A const T& parameter aliasing old top becomes dangling when old storage is released. Noexcept relocation does not make that reference valid again.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Array stack operations and object lifetime (continued)

Charge snapshot construction separately; constant amortization for the complete call assumes fixed-cost element operations. Noncopyable T does not support this copying use of push(top()). If parameter copying fails, the body has not started, so size,capacity,and existing objects are unchanged. If new-element construction fails after successful growth, this pseudocode keeps size unchanged and old values valid, but capacity and prior references may already have changed; it does not promise the strong guarantee for every complete push.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

The FIFO Queue ADT

- The abstract state is again $[a_0, \ldots, a_{n-1}]$. `enqueue(x)` appends, while `dequeue()` removes the first element.
- Shifting all remaining array elements after each front removal makes a dequeue cost $\Theta(n)$.
- A circular array preserves logical order without moving every remaining element.
- FIFO specifies return order. It guarantees neither thread safety nor equal execution time for jobs.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circular queue representation invariant

- For capacity `C>0`, store head and size. Logical element k occupies `(head+k) mod C`.
- Empty means size=0 and full means size=C. No unused slot is needed to distinguish ambiguous equal head and tail indices.
- Enqueue writes at `(head+size) mod C`, then increments size. Dequeue reads head, sets `head=(head+1) mod C`, then decrements size.
- Each operation takes worst-case Θ(1) with fixed capacity and constant-time element movement. Resizing copies in logical order and can take Θ(n) for one operation.

Lin Cing https://github.com/xinhi6716/computer-sci elf-study

Worked circular trace

With capacity 4, enqueue A,B,C, dequeue twice, then enqueue D,E,F.

State	head	size	Logical sequence
Enqueue ABC	0	3	A,B,C
Dequeue twice	2	1	C
Enqueue D,E	2	3	C,D,E
Enqueue F	2	4	C,D,E,F

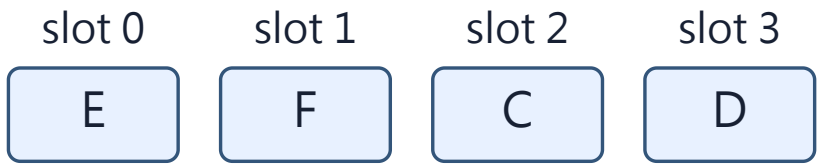
Physical slots 0,1,2,3 finally contain E,F,C,D.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

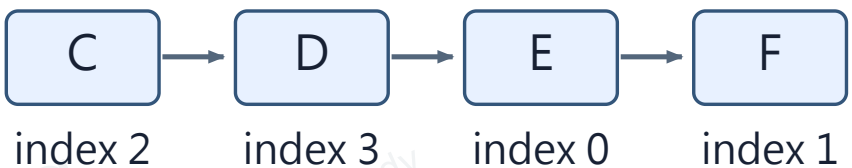
Worked circular trace (continued)

Physical [E,F,C,D] versus FIFO [C,D,E,F]

Physical slots



Logical order / physical indices



head=2; size=4; C=4

indices: 2 → 3 → 0 → 1

slot(k)=(head+k) mod C

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete circular-queue operations and arithmetic bounds

Use head and size with C>0. This fixed-capacity version returns full instead of overwriting the oldest value. Value movement costs and exception behavior belong to the element contract.

```
slot(k):
    require 0<=k<C
    return head+k if k<C-head else k-(C-head)
try_enqueue(x):
    if size==C: return full
    construct_at(A+slot(size),x)
    size=size+1; return success
front(): require size>0; return reference(A[head])
dequeue():
    require size>0
    value=read_or_move(A[head])
    destroy_at(A+head)
    head=0 if head==C-1 else head+1
    size=size-1
    return value
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete circular-queue operations and arithmetic bounds (continued)

The branched slot formula avoids overflowing head+k. Enqueue first checks nonfull, so k=size<C. After the last removal, head may be any valid slot. The abstraction map `(head+k) mod C` proves correctness: enqueue appends exactly the new last value, and dequeue maps old logical k+1 to new logical k. Both have worst-case Θ(1) fixed-capacity work.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Circular growth must restore logical order

Grow physical `[E,F,C,D]`, head=2,size=4,C=4 to C′=8. Copying the old slots unchanged and retaining head=2 is wrong: the new modulus sends the expected E position to unconstructed slot 4.

Logical k	Old slot (2+k) mod 4	Value	New slot
0	2	C	0
1	3	D	1
2	0	E	2
3	1	F	3

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

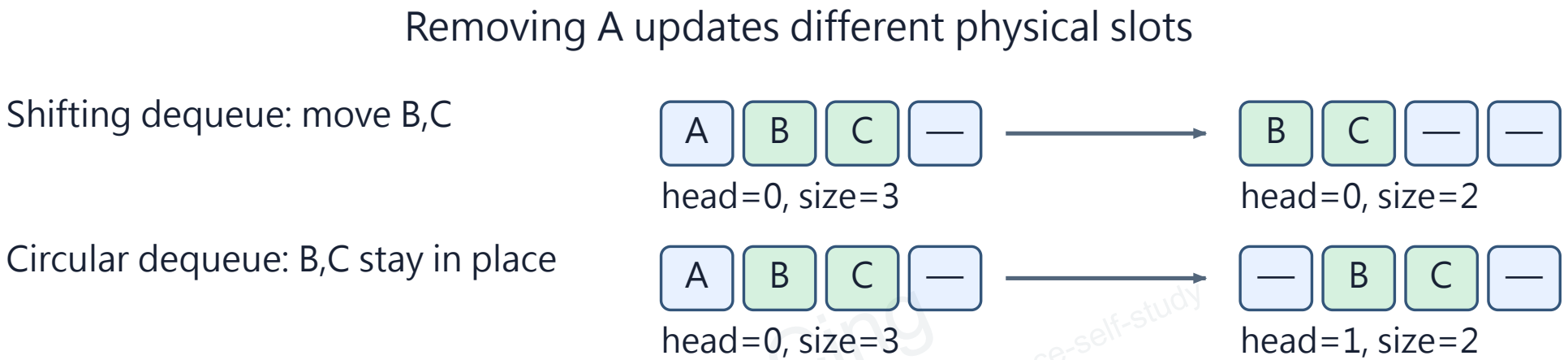
```
allocate new capacity C'
for k=0..size-1: construct new[k] from old[slot(k)]
commit new storage; head=0; C=C'; destroy/release old storage
```

Circular growth must restore logical order (continued)

The logical result is C,D,E,F. Append G at new slot 4. One growth costs Θ(size). Doubling only on full gives amortized Θ(1) enqueue from empty by the geometric movement sum. This does not satisfy fixed-capacity worst-case constant latency.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Dequeue movement and empty/full states



Capacity 4; — means no live item. Same head=0: size=0 is empty, size=4 is full.

With capacity 4, empty and full can share the same head. Size or another explicit convention distinguishes them. A shifting array moves every remaining value; the circular version advances head and destroys the old front object. Both preserve FIFO values but have different costs.

Maze states and legal moves

- Passable cells are vertices, with edges between adjacent passable cells. This example permits four directions and excludes diagonals.
- Search state includes the current cell, visited information, and predecessors for path reconstruction.
- DFS can use a stack. It finds a path but does not generally minimize the number of steps.
- If all moves cost 1, BFS uses a queue. The first discovery level of the exit gives the shortest number of steps.

Lin Cing
https://github.com/xinhi6716/computer-science-self-stu

DFS backtracking in an original maze

The symbol # is a wall. Coordinates start at the upper-left cell `(0,0)`.

```
S . # .
# . # .
. . . G
```

- Use neighbor order right, down, left, up. From `S=(0,0)`, one path is `(0,1)→(1,1)→(2,1)→(2,2)→G`.
- A stack frame may store `(cell,next_direction)`. At a dead end, pop the frame and resume untried directions from the preceding cell.
- Visited marks prevent cycles. Enumerating all simple paths requires a different policy for marks during backtracking.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Resumable maze DFS frames

A frame `(u,next_direction)` stores where recursive neighbor scanning will resume. Mark visited on first push.

```
visited[start]=true; parent[start]=none; push((start,0))
while stack is not empty:
    (u,i)=top()
    if u==goal: return parent path
    if i==4: pop(); continue
    top().next_direction=i+1
    v=neighbor(u,i)                  // right, down, left, up
    if v is in bounds, passable, and not visited[v]:
        visited[v]=true; parent[v]=u; push((v,0))
return unreachable
```

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Resumable maze DFS frames (continued)

At (2,1), right neighbor (2,2) is pushed first, then (2,3)=G is found. The path contains five edges. Removing the goal test while retaining passability gives complete discovery order `(0,0),(0,1),(1,1),(2,1),(2,2),(2,3),(1,3),(0,3),(2,0)`. Each frame tries at most four directions. Finite cells and one first push per cell prove termination. Parent edges strictly reverse discovery time, so they are acyclic and reconstruct a legal path.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Maze BFS distances and predecessors

Use the same maze. Assign dist and parent at enqueue time. Unit edge costs give final distances:

```
dist[start]=0; parent[start]=none; enqueue(start)
while queue is not empty:
    u=dequeue()
    for v in legal_four_neighbors(u):
        if v is undiscovered:
            dist[v]=dist[u]+1; parent[v]=u; enqueue(v)
```

```
0  1  #  7
#  2  #  6
4  3  4  5
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Maze BFS distances and predecessors (continued)

Dequeued cell	New cells and distances	Queue afterward
(0,0)	(0,1):1	[(0,1)]
(0,1)	(1,1):2	[(1,1)]
(1,1)	(2,1):3	[(2,1)]
(2,1)	(2,2):4, (2,0):4	[(2,2),(2,0)]
(2,2)	(2,3):5	[(2,0),(2,3)]
(2,0)	None	[(2,3)]
(2,3)	(1,3):6	[(1,3)]
(1,3)	(0,3):7	[(0,3)]
(0,3)	None	[]

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Maze BFS distances and predecessors (continued)

Follow G's parents through `(2,2),(2,1),(1,1),(0,1),(0,0)` and reverse to obtain the five-edge shortest path. Earlier processing of all smaller layers proves optimality. Merely selecting a queue does not prove a correct BFS implementation.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Search correctness and cost

- Every explored cell is reachable from the start, and each predecessor edge leads to an earlier discovery.
- If DFS tries every legal neighbor and has no pending frames, it has covered every reachable cell. An undiscovered exit is unreachable.
- With V passable cells and E adjacency edges, adjacency-list DFS and BFS take $\Theta(V+E)$ for a complete traversal. Marks and predecessors use $\Theta(V)$ space.
- A four-neighbor grid has $E=O(V)$, giving linear work. Copying a complete path at every step can violate that implementation bound.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Expression representations and grammar

- Infix `a+b*c`, prefix `+ a * b c`, and postfix `a b c * +` describe the same expression tree.
- Initially restrict the grammar to binary operators, integer tokens, parentheses, and explicit precedence. A negative numeric token and a unary minus operator are different syntax choices.
- Infix requires precedence and associativity. Default left-associative `a-b-c` means `(a-b)-c`.
- Whitespace cannot reliably tokenize every programming language. A complete compiler performs lexical analysis before parsing.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Operand order in postfix evaluation

1. Scan tokens left to right. Push numeric values.
2. For a binary operator, pop the right operand b first, then the left operand a. Push the result of `a op b`.
3. Reject if fewer than two values are available. At the end exactly one value must remain.

For `8 3 2 * -`, the stacks are `[8]`, `[8,3]`, `[8,3,2]`, `[8,6]`, `[2]`. Division additionally requires zero-divisor and arithmetic-range checks.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Prefix evaluation and linear time

- Scan prefix right to left. At an operator, the first popped value is the left operand and the second is the right operand.
- For `- 8 * 3 2`, first compute $3 \times 2 = 6$, then $8 - 6 = 2$.
- Each token is pushed and popped at most a constant number of times. Infix-to-postfix conversion and prefix or postfix evaluation therefore take O(t), for t tokens.
- Auxiliary stack space is O(t) in the worst case. Fixed-range arithmetic is treated as constant time. Arbitrary-precision arithmetic needs a separate cost model.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete infix-to-postfix conversion

The grammar contains nonnegative integer tokens, binary `+ - * / ^`, and parentheses. Here ^ denotes right-associative exponentiation, not C++ XOR. Other operators associate left. Precedence is ^, then */, then +-. Unary minus is excluded from this grammar.

```
output = []; ops = []; need_operand = true
for token in tokens:
    if token is a number:
        require need_operand; output.append(token); need_operand=false
    else if token == '(':
        require need_operand; ops.push('(')
    else if token == ')':
        require not need_operand
        while ops is not empty and ops.top != '(': output.append(ops.pop())
        require ops is not empty; ops.pop()   // discard matching left parenthesis
        need_operand=false
    else if token is binary operator op:
        require not need_operand
```

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete infix-to-postfix conversion (continued)

```
        while ops is not empty and ops.top != '(' and
              (prec(ops.top)>prec(op) or
               (prec(ops.top)==prec(op) and op is left associative)):
            output.append(ops.pop())
        ops.push(op); need_operand=true
    else: reject
require not need_operand                    // EOF: reject empty or incomplete input
while ops is not empty:
    require ops.top != '('                  // reject unmatched left parenthesis
    output.append(ops.pop())
return output
```

The loop must repeatedly pop. One if statement misses cases where several stacked operators must be emitted. EOF requires a separate drain and parenthesis check. Every failed requirement rejects the input instead of returning a partial expression.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

EOF draining is required

`9 - 4`

Token	Operator stack, bottom to top	Output
`9`	`∅`	`9`
`-`	`-`	`9`
`4`	`-`	`9 4`
`EOF`	`∅`	`9 4 -`

Before EOF the output is only `9 4`. Draining gives `9 4 -`, with value 5. Prefix is `- 9 4`.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete parenthesized scan

`(9-4)*(2+3)`

Token	Operator stack, bottom to top	Output
(	(	∅
9	(	9
-	(-	9
4	(-	9 4
)	∅	9 4 -
*	*	9 4 -
(	* (	9 4 -
2	* (	9 4 - 2

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete parenthesized scan (continued)

Token	Operator stack, bottom to top	Output
`+`	`* ( +`	`9 4 - 2`
`3`	`* ( +`	`9 4 - 2 3`
`)`	`*`	`9 4 - 2 3 +`
`EOF`	`∅`	`9 4 - 2 3 + *`

Postfix stack states are `[9]→[9,4]→[5]→[5,2]→[5,2,3]→[5,5]→[25]`. Prefix `* - 9 4 + 2 3` also evaluates to 25.

Lin Cing https://github.com/xinhi6716/computer-science-self-study

Repeated popping for consecutive operators

`9-4*2/1+3`

Token	Operator stack, bottom to top	Output
9	∅	9
-	-	9
4	-	9 4
*	- *	9 4
2	- *	9 4 2
/	- /	9 4 2 *
1	- /	9 4 2 * 1
+	+	9 4 2 * 1 / -

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Repeated popping for consecutive operators (continued)

Token	Operator stack, bottom to top	Output
3	+	9 4 2 * 1 / - 3
EOF	∅	9 4 2 * 1 / - 3 +

At +, pop / and then the equal-precedence left-associative -. Popping once is insufficient. The value is 4. For right-associative 2^3^2, do not pop on the second ^. Postfix is 2 3 2 ^ ^, giving 512 rather than 64.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Rejection points for invalid token sequences

Input	Rejection point	Reason
`(9-4`	EOF	A left parenthesis remains after emitting -
`9-4)`	At)	No matching left parenthesis
`9-`	EOF	need_operand remains true
`9 4`	Second number	Missing binary operator
`()`	At)	Missing operand inside parentheses

Do not infer unary versus binary syntax from stack size. Evaluation separately rejects division by zero and fixed-integer overflow.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Operator-stack invariant and termination proof

An unmatched left parenthesis separates stack segments. Within each segment, precedence is nondecreasing from bottom to top. Equal-precedence operators can remain together when waiting for right association. Output contains the postfix order of completed subexpressions; stacked operators await their right operand or a higher-precedence subexpression.

Repeated popping emits exactly the operators that must finish before the new operator. A right parenthesis finishes its entire local segment while preserving outside operators. At EOF no further operand can arrive, so every pending operator must be emitted. A remaining left parenthesis has no valid mate and must cause rejection, not appear as an output operator.

Each operator and parenthesis is pushed and popped at most once. A single token can trigger multiple pops, but the complete t-token conversion uses O(t) time and O(t) space. This is an aggregate proof, not constant worst-case work per token. Unary operators, functions, negative numeric tokens, and special floating-point values require an expanded grammar and proof.

Sources and teaching boundaries

Exercises and worked answers are separate chapter files.

Standard interfaces were checked against C++ stack and queue. The MIT 6.006 2020 index, L9–L10, provides search-course context. Maze and expression examples are original.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study